11-411/11-611 Natural Language Processing
Homework 3

Due on Thursday, April 02, 2026, 11:59 PM

Introduction

In this homework, you will be experimenting with fine-tuning sequence-to-sequence transformer models for Neural Machine Translation (NMT). NMT requires your model to generate an output sequence. Specifically, a French translation of an English sentence.

The task is English-to-French translation. Here is an example pair from the dataset::

> **English**: 'Some fine mansions here and there made magnificent outlines against the picturesque attics of the left bank. The house of Nevers, the house of Rome, the house of Reims, which have disappeared; the Hôtel de Cluny, which still exists, for the consolation of the artist, and whose tower was so stupidly deprived of its crown a few years ago.'
>
> **French**: 'Quelques beaux hôtels faisaient aussi çà et là de magnifiques saillies sur les greniers pittoresques de la rive gauche, le logis de Nevers, le logis de Rome, le logis de Reims qui ont disparu ; l'hôtel de Cluny, qui subsiste encore pour la consolation de l'artiste, et dont on a si bêtement découronné la tour il y a quelques années.'

The goal of this assignment is to fine-tune pre-trained MarianMT models for English-to-French translation, track your experiments using wandb, and perform an in-depth error analysis using BLEU score as your primary metric.

Learning Objectives

Sequence-to-sequence transformer models are the backbone of modern machine translation and generation systems. In this assignment, you'll gain hands-on experience with:

- Fine-tuning a pre-trained encoder-decoder model (MarianMT) for machine translation.
- Understanding the difference between encoder-only models (BERT) and encoder-decoder models (Marian).
- Using BLEU score for evaluating generation quality.
- Analyzing translation errors and understanding model failure modes.
- Comparing model size vs. performance trade-offs

Preliminaries

The starter notebook trains a MarianMT model on the OPUS Books English-French dataset available on Hugging Face (Helsinki-NLP/opus_books, en-fr subset).

The dataset is split into:

- **Training Set:** ~127K sentence pairs
- **Validation Set:** ~2K sentence pairs (sampled from the training data)
- **Test Set:** ~1k English sentences (test-DIST.json)

You can run this on Google Colab, on Kaggle or on your machine (just using the CPU and no GPU), though the latter would be significantly/painfully slower. If you have access to an Apple Silicon MacBook Pro, you can also use its GPU, which runs about 3-4 times faster than the CPU but also slower than a Colab Nvidia GPU. Likewise, if you have a GPU on your PC laptop, you can use it for this assignment. However, you will be on your own with respect to support.

If Google Colab claims you have used too much GPU you can run it on AWS SageMaker. You can also connect to a Jupyter Notebook from an AWS EC2 instance. You can find support for doing this online. The tutorial here is a good place to get started but is outdated in certain details.

Task 1: Model Selection – Comparing MarianMT Models

Introduction to Models:

1. **Helsinki-NLP/opus-mt-en-fr (Small Model):**
   A compact MarianMT model trained on OPUS parallel corpora for English-to-French translation. It uses a standard encoder-decoder transformer architecture and is fast to fine-tune.
2. **Helsinki-NLP/opus-mt-tc-big-en-fr (Big Model):**
   A significantly larger MarianMT model with more encoder/decoder layers and wider hidden dimensions. It generally achieves higher BLEU scores on standard benchmarks but takes longer to fine-tune.

Objective:

Compare both models using the same baseline configuration. Evaluate on the validation set using BLEU score after each epoch.

Experimentation:

The starter code, after completion, trains for n epochs, and at the end of each epoch, runs the model on the validation data set, reporting loss and accuracy. At the end of training, it runs the final model on the default test set and reports test loss and test accuracy.

Note: You might find the PyTorch Documentation helpful in completing some of the functions.

Results

Part 1: Results of the Initial Experimentation

Model	Val BLEU Score	Validation Loss
Helsinki-NLP/opus-mt-en-fr		
Helsinki-NLP/opus-mt-tc-big-en-fr		

Part 2: wandb Visualizations

- **Instructions:**
  Add plots from wandb that show training and validation curves (loss and accuracy over epochs). These visualizations should help compare how each model learns over time.

#TODO: Replace this with your plots

Part 3: Model Selection & Analysis

- **Select the Better Model:**
  Based on validation BLEU, pick the better-performing model.

#TODO: Add the answer here

- **Discussion:**
  Discuss why you believe the selected model outperforms the other. Consider convergence speed, training stability, and BLEU score trajectory. Also comment on the tokenization differences between the two models, how does each model tokenize the same French sentence?

#TODO: Add the answer here

Task 2: Hyperparameter Tuning and Experimentation

Objective:

Using the best-performing model from Task 1, experiment with various hyperparameters to further improve translation quality on the English-to-French task

Experimentation:

We would like you to experiment with alternate classifier configurations to see if you get better or worse results on the test set.

Parameters to Tune:

- **Learning Rate:** Translation fine-tuning is sensitive to learning rate.
- **Batch Size:** Larger batches are faster but may reduce gradient signal quality.
- **Number of Epochs:** Watch for overfitting. BLEU on the validation set may plateau or drop.
- **Beam Search Width:** During generation, try different beam sizes. This affects inference quality, not training.
- **Freezing the encoder:** Experiment with freezing the encoder entirely vs. fine-tuning it.
- **Label Smoothing**: Optionally, try adding label smoothing to the cross-entropy loss to improve generalization.

Note: Batch sizes larger than 32 may cause Colab to run out of GPU memory. You may use gradient accumulation to simulate larger effective batch sizes if desired.

Results

Part 1: Results of the Initial Experimentation

Configuration Table

Config	Epochs	beam	Label smooth	learning rate	freeze base	batch size
Baseline	5					
Config 1	5					
Config 2	5					
Config 3	5					
Config 4	5					
Config 5	5					

Result Table

Config	Train Loss	Val Loss	Val BLEU
Baseline			
Config 1			
Config 2			
Config 3			
Config 4			
Config 5			

Part 2: wandb Visualizations

- **Instructions:**
  Add plots from wandb that show training and validation BLEU for all your experiments. Make sure your legend indicates what configuration each line denotes

#TODO: Replace this with your plots

Part 2: Analysis of Hyperparameter Trends

After presenting your results table and wandb plots, provide a discussion analyzing the trends observed across different hyperparameter configurations. Your analysis should include:

Trend Analysis:
Discuss how does learning rate, batch size, beam size, and encoder freezing affect BLEU score? Are there configurations that consistently improve results?

Trade-offs:
Address any trade-offs between training speed, memory usage, and model performance. Did freezing the encoder significantly hurt performance? Did larger beam sizes always help at inference time?

Observations:
Comment on specific improvements or degradations. For instance, why might a very high learning rate cause BLEU to collapse? What happened when you changed beam size without changing training at all?

#TODO: Add your answer here

Task 3: Final run on our test set

When you have selected the final model you want to use, you should run it one last time on the separate test set `test-DIST.json` file we have provided, and run the function so that your classification results are saved to a file `test-translations.txt` one translated sentence per line, in the same order as the input file.

One important component of all NLP tool or solution development work is **error analysis**. Error analysis is a critical step in understanding where your model makes mistakes and why. By analyzing false positives and false negatives, you can gain insights into potential improvements. For machine translation, this is especially rich because errors can stem from vocabulary gaps, grammatical disagreement, word order differences between English and French, or handling of named entities

Here is how we want you to proceed:

1. Retrieve a sample of 20–30 examples from the validation set where your model produced low sentence-level BLEU scores (worst translations).
2. Retrieve another sample of 20–30 examples where the model produced high sentence-level BLEU scores (best translations).
3. For the worst translations, examine the source, reference, and hypothesis manually. Categorize the errors, are they due to unknown vocabulary, wrong verb tense, incorrect gender/number agreement, or hallucinations (fluent but wrong content)?
4. For the best translations, note what characteristics these sentences share, are they short, simple, or from a specific domain?
5. Comment on any patterns you observe in sentence length vs. BLEU score. Does the model struggle more with longer sentences?
6. Suggest at least two concrete improvements based on your findings. These could be data augmentation strategies, architectural changes, or post-processing steps

Present your findings, and based on your findings, suggest possible improvements! Feel free to be as creative as you can!

#TODO: Add your answer here

Submission Guidelines

When you're ready to submit your assignment, please follow these steps:

1. Submit the completed ipynb **without clearing any output cells**, renamed as `hw3.ipynb`
    - Make sure your code can load the best model configuration
    - Include necessary functions for tokenization, prediction, etc.
    - Make sure it can run the test predictions on `test-DIST.json`

2. Download `test-translations.txt` the notebook generated with your predictions on the provided test set. Perform a manual check to see if:
    - Each line should contain one translated sentence per line, in the same order as the input file.
    - No other content should be in this file

3. Read and complete the hw3 report (report.pdf). It has the following sections:

- ***Part 1: Model Selection***
    - Compare performance of both models
    - Explain your choice of model and why it performed better
    - Discuss any observations about tokenization differences

- ***Part 2: Hyperparameter Tuning***
    - Table with all configurations tested and their results
    - Analysis of how different hyperparameters affected performance
    - Include and explain wandb plots
    - Justify your final model configuration

- ***Part 3: Final run and Error Analysis***

Submit the following files to Gradescope:
- `hw3.ipynb`: Your code implementation
- `test-translations.txt`: Predictions on the test set
- `report.pdf`: Your detailed report

Make sure your name and Andrew ID are on the first page of your report.

Scoring Guidelines

1. Your writeup - report.pdf (40%)
2. Programming Assignment - hw3.ipynb and test-translations.txt (60%)